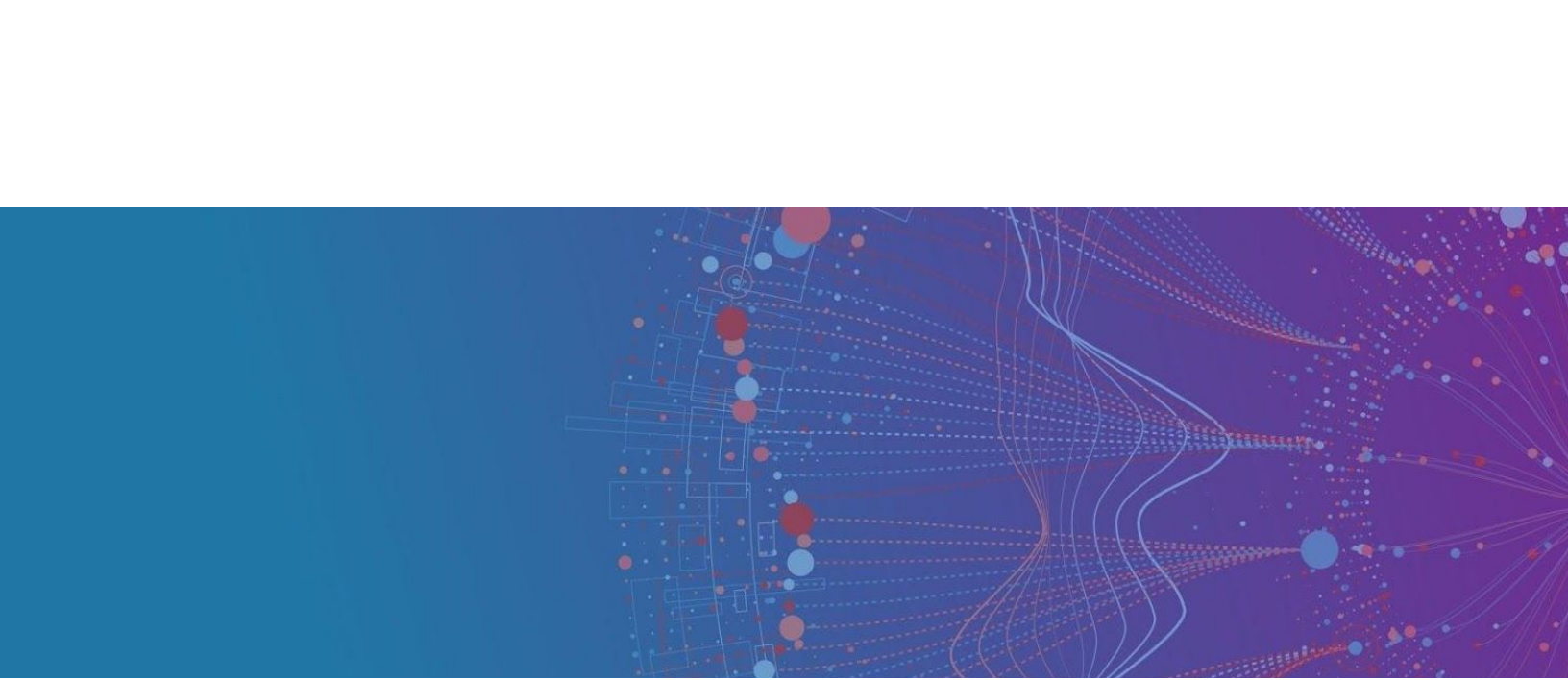




Data Science Career Track

Capstone Three Step 2: Data Wrangling

Estimated Time: 6-12 Hours

Overview

Welcome to the data wrangling stage of your third capstone project. In addition to the data wrangling steps you applied to your previous capstone projects, you now need to address some unique characteristics related to the nature of the Business Insider track. The exact steps depend heavily on the **nature of the business problem** and the **type of data** you need to deal with for your third capstone project.

As a general definition, consider “Data Wrangling” as the set of tasks that need to be implemented to take the “raw” data and transform them so that they are ready to be further analyzed.

Tips: Ask your mentor if you need help to determine which steps to take to get your specific datasets ready for further analysis downstream.

Outputs for this step:

1. A Jupyter notebook you developed. Make sure your code is well-documented, structured, and that you include Markdown cells to explain your rationale in detail, and to guide the readers of your notebook.
2. “Cleansed” datasets. This means that after the original datasets have been processed through the tasks discussed above, the resulting “cleansed” datasets are persisted using methods such as: creating CSV files, creating “serialized” files, database tables, etc.

During your next mentor call, discuss other approaches that might be appropriate for the type of data you're working with for your project.

Samples for this step:

1. Sample 1: [Sinead O'Connor's Wrangling notebook](#)
2. Sample 2: [Neha's Wrangling notebook](#)

Project Guidance

Below are some general points you might want to consider as you wrangle data for your third capstone. Ask your mentor to play the role of the intended client when considering alternatives that naturally would call for you to "consult with your client".

1. Data Acquisition

Determine if you need to implement procedures in Python and/or SQL to acquire the datasets with minimal manual intervention. The general "rule of thumb" is that manual interventions can be laborious, error-prone, and not scalable — and therefore must be avoided whenever possible (and it is almost always possible!). Examples include:

- Deriving dates from file names: to avoid having to hard-code dates in the original dataset
- Translating numerical codes to interpretable codes: to avoid having to manually translate large amounts of codes
- Generating datasets by using complex SQL queries on complex tables
- In general: how to process raw data assisted with parameters defined in a configuration file

2. Data quality

Determine if it is possible to formally define rules that certain types of data must abide by. These rules typically fall into two general classes: syntactic rules and semantic rules. The first class of rules deals with the format of the data. The second class deals with the intended meaning of the values in the context of the business problem. **For example:**

- Syntactic:
 - Social security numbers must follow a specific format: XXX-XX-XXXX
 - Zipcodes must be 5 characters-long, which may include leading zeros
- Semantic:
 - Can sales, costs, and units be negative or zero? What implications would that have?

- How to differentiate cities with the same name, for instance, by creating qualifying state names such as “Troy-NY” or “Troy-MI”?
- How to match names that belong to the same person, but are spelled in different ways? (This happens frequently in healthcare applications.)
- More generally, how to check if numerical non-categorical values fall into prescribed boundaries, and categorical values take expected values? (Note: discuss with your mentor the dangers of automatically deleting “outliers” regardless of the underlying business application.)

3. Dealing with missing values

Recall that null values indicate that values are missing, but not all missing values are null. Therefore, we can consider the following cases. Discuss with your mentor if these cases cover all possibilities in the context of your specific project.

- Marking all missing values as null (NaN in Python)
- Dropping an entire column if the percentage of missing values is larger than a threshold determined by the specific business problem. Also, consider dropping an entire row when this percentage is small.
- For variables that take **non-categorical values**, consider the following methods, in collaboration with your mentor, in the context of the business problem:
 - Study the distribution of the underlying variable and determine the appropriate method to impute missing values (like mean, median, mode).
 - Apply regression methods to impute missing values.
- For variables that take **categorical values**, consider the following approaches:
 - Leave the value as NaN, and use a method that treats NaN as a value by building “dummy variables” - for instance Panda’s ‘get_dummies’.
 - Introduce a specific default value whose meaning is “not known.” Computationally, this would get the same effect as the previous approach, but it is a little more interpretable.
 - Impute the most frequent value in the distribution

4. Dealing with small datasets

There is no universal definition of what a “small” dataset is. Most of the time you’ll know when the dataset seems small. Remedies include an augmentation and/or resampling approaches that aligns with the specific nature of the dataset and business problems.

5. Dealing with large, very large, and big datasets

- Large datasets: A dataset is “large” if it is not practical or possible to load it all at once to the main memory for processing.

- A dataset with 40 million records that can be processed in 4 batches of 10 million records each, and each batch takes 20 minutes to process
- Very large datasets: On the other hand, a dataset is 'very large' if it is not practical or possible to process it in batches using the resources at hand.
 - A dataset with 100 million records it could take days to process 10 batches of 10 million records each.
- Big datasets: Finally, a dataset is considered 'big' if it cannot be stored and processed with the available resources.
 - A dataset with 1000 million records might need several weeks to process 100 batches of 10 million records each.

If you find it necessary to work with a large, very large, or big dataset, consider the following:

- Discuss with your mentor the advantages and disadvantages of this approach, and how to implement it correctly from a statistical standpoint.
- Implement an approach to acquire all the information contained in the dataset using batch processing.
- Use a 'big data' distributed infrastructure such as the ones offered by Amazon, Google, Microsoft, or IBM
- Depending on the size of the dataset, you might need to combine the three approaches above.

6. **Blending multiple datasets**

It is very common for data science projects to deal with multiple datasets. For instance, consider operational retail data that must be blended with demographic data associated with customers, and weather data associated with their locations. In these cases, all the activities mentioned in the previous points might apply to each dataset, plus implementing the blendings might trigger additional complex tasks.